Modul XII - Akses Lokasi dengan GPS di Flutter

12.1. Ringkasan

Komponen	Uraian
Tujuan Praktikum	1. Memahami konsep pengambilan lokasi perangkat menggunakan GPS pada Flutter. 2. Mengakses koordinat *latitude* dan *longitude* perangkat secara real-time. 3. Mengubah koordinat menjadi informasi alamat (*reverse geocoding*) seperti kelurahan, kecamatan dan kota. 4. Menangani izin (*permission*) lokasi pada Android. 5. Menampilkan informasi lokasi pada tampilan aplikasi Flutter secara interaktif dan responsif.
Dasar Teori	1. GPS (*Global Positioning System*) adalah sistem navigasi berbasis satelit yang memungkinkan perangkat untuk menentukan koordinat geografis (*latitude* dan *longitude*). 2. Dalam Flutter, paket *geolocator* digunakan untuk mengambil lokasi perangkat, memeriksa status layanan GPS, dan mengelola izin lokasi. 3. Paket *geocoding* digunakan untuk melakukan *reverse geocoding*, yaitu mengubah koordinat (*latitude* & *longitude*) menjadi alamat yang dapat dibaca manusia, seperti kecamatan, kota, atau negara. 4. *Permission Handling* sangat penting karena Android meminta izin eksplisit dari pengguna untuk mengakses lokasi. Terdapat beberapa status izin: *denied*, *deniedForever*, dan *granted*. 5. Menampilkan data lokasi pada UI memerlukan *State Management*, dalam modul ini menggunakan *StatefulWidget* dengan metode *setState()* untuk memperbarui tampilan saat data lokasi diperoleh atau saat terjadi *error*.
Alat & Bahan	1. Perangkat Lunak: - Flutter SDK (versi stabil terbaru) - Android Studio atau VS Code - Emulator Android atau perangkat fisik dengan GPS aktif 2. Paket Flutter: - geolocator: ^7.7.0 - geocoding: ^3.0.0 - google_fonts: ^6.1.0 3. Konfigurasi Android Manifest: - `<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />` - `<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />`

12.2. Langkah Kerja

a. Buat projek baru dengan nama *lokasi_flutter*.

b. Struktur folder awal projek seperti berikut:

lokasi_flutter/

```
├── android/
├── ios/
├── lib/
│   └── main.dart
└── pubspec.yaml
```

c. Buka file *pubspec.yaml* lalu tambahkan paket yang dibutuhkan di bawah bagian dependencies seperti berikut:

```
dependencies:
  flutter:
    sdk: flutter
  google_fonts: ^6.1.0
  geolocator: ^7.7.0
  geocoding: ^3.0.0
```

Simpan file *pubspec.yaml* lalu jalankan perintah *flutter pub get* di terminal agar paket terintegrasi dengan proyek.

d. Buka file *android/app/src/main/AndroidManifest.xml* lalu tambahkan izin lokasi berikut setelah baris <*manifest*> dan sebelum tag <*application*>:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

e. Buka file lib/main.dart, kemudian import paket yang dibutuhkan:

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'package:geolocator/geolocator.dart';
import 'package:geocoding/geocoding.dart';
```

f. Buat kelas *MyApp* menggunakan *MaterialApp* dan jadikan *LocationScreen* sebagai halaman utama aplikasi.

```
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Lokasi Saya',
      theme: ThemeData(
        primarySwatch: Colors.indigo,
        textTheme: GoogleFonts.poppinsTextTheme(
          Theme.of(context).textTheme,
        ),
      ),
      home: const LocationScreen(),
    );
  }
}
```

g. Buat kelas *LocationScreen* sebagai turunan *StatefulWidget* dan tambahkan variabel untuk menyimpan data lokasi, status loading, dan pesan error.

```
class GeolocationScreen extends StatefulWidget {
  const GeolocationScreen({super.key});

  @override
  State<GeolocationScreen> createState() => _GeolocationScreenState();
}

class _GeolocationScreenState extends State<GeolocationScreen> {

  String? _kecamatan;
  String? _kota;
  bool _isLoading = false;
  String? _errorMessage;

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

h. Buat fungsi *_getLocation()* di dalam kelas *_GeolocationScreenState* sebelum method *build()*. Fungsi ini bertugas untuk mengambil koordinat GPS perangkat, memeriksa izin akses lokasi, dan melakukan *reverse geocoding*. Hasilnya berupa informasi alamat yang dapat dibaca, seperti kecamatan/kelurahan dan kota, yang kemudian ditampilkan di layar aplikasi.

Di awal fungsi *setState()* digunakan untuk memberitahu Flutter bahwa tampilan perlu diperbarui. Pada tahap ini, aplikasi menandai bahwa sedang memproses lokasi dengan *_isLoading = true* dan membersihkan data lama sehingga informasi yang ditampilkan tidak salah atau tertinggal dari proses sebelumnya. Selanjutnya aplikasi mengecek apakah layanan lokasi atau GPS aktif menggunakan *Geolocator.isLocationServiceEnabled()*. Jika layanan tidak aktif pengguna akan diberi pesan untuk menyalakan GPS terlebih dahulu.

Setelah itu, aplikasi memeriksa izin akses lokasi melalui *Geolocator.checkPermission()*. Jika izin belum diberikan, aplikasi akan meminta izin dengan *Geolocator.requestPermission()*. Izin lokasi bisa ditolak sementara (*denied*) atau permanen (*deniedForever*). Jika ditolak permanen, pengguna perlu diarahkan untuk mengubah pengaturan aplikasi agar lokasi dapat diakses.

Jika izin diberikan, aplikasi akan mengambil posisi perangkat saat ini menggunakan *Geolocator.getCurrentPosition()* yang menghasilkan koordinat latitude dan longitude dengan presisi tinggi. Koordinat ini kemudian diubah menjadi alamat yang bisa dibaca manusia melalui fungsi *placemarkFromCoordinates()*. Hasilnya berupa daftar *Placemark* dimana aplikasi mengambil data pertama (*placemarks[0]*) untuk menampilkan kecamatan (*subLocality*) dan kota (*subAdministrativeArea*).

Selama proses berlangsung, semua kemungkinan error ditangani di blok *catch* agar aplikasi tidak crash dan pengguna tetap mendapatkan pesan yang jelas jika terjadi masalah. Terakhir blok finally memastikan status loading atau spinner berhenti ditampilkan baik proses berhasil maupun gagal sehingga tampilan aplikasi selalu responsif dan informatif.

```
Future<void> _getLocation() async {
  // 1. Set status loading dan reset data
  setState(() {
    _isLoading = true;        // Menandai aplikasi sedang memproses lokasi
    _errorMessage = null;     // Menghapus pesan error sebelumnya
    _kecamatan = null;        // Menghapus data kecamatan sebelumnya
    _kota = null;             // Menghapus data kota sebelumnya
  });

  try {
    // 2. Cek apakah layanan lokasi (GPS) aktif
    bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
      // Jika GPS tidak aktif, tampilkan pesan error
      throw Exception('Layanan lokasi tidak aktif. Mohon aktifkan GPS.');
    }

    // 3. Periksa izin akses lokasi
    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      // Jika izin belum diberikan, minta izin kepada pengguna
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied) {
        throw Exception('Izin lokasi ditolak oleh pengguna.');
      }
    }
    if (permission == LocationPermission.deniedForever) {
      // Jika izin ditolak permanen, beri instruksi untuk mengubah pengaturan
      throw Exception(
          'Izin lokasi ditolak permanen. Anda harus mengubahnya di pengaturan
aplikasi.');
    }

    // 4. Ambil posisi perangkat saat ini
    Position position = await Geolocator.getCurrentPosition(
        desiredAccuracy: LocationAccuracy.high); // Presisi tinggi

    // 5. Lakukan reverse geocoding untuk mengubah koordinat menjadi alamat
    List<Placemark> placemarks =
        await placemarkFromCoordinates(position.latitude, position.longitude);

    if (placemarks.isNotEmpty) {
      // Ambil data lokasi pertama dari hasil reverse geocoding
      Placemark place = placemarks[0];
      setState(() {
        _kecamatan = place.subLocality;  // subLocality biasanya kecamatan
        _kota = place.subAdministrativeArea; // subAdministrativeArea biasanya
kota
      });
    } else {
      // Jika tidak ada data alamat yang ditemukan
      throw Exception('Tidak dapat menemukan informasi alamat.');
    }
  } catch (e) {
    // 6. Tangani error dan tampilkan pesan
    setState(() {
      _errorMessage = e.toString().replaceAll("Exception: ", "");
    });
  } finally {
    // 7. Hentikan status loading
    setState(() {
      _isLoading = false;
    });
  }
}
```

i. Buat tampilan UI di method *build()* menggunakan *Scaffold*, *Card*, dan *ElevatedButton*. Gunakan logika *_isLoading*, *_errorMessage*, dan data lokasi untuk menampilkan konten yang sesuai.

```
@override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text(
          'Lokasi Saya',
          style: TextStyle(
            color: Colors.white,
          ),
        ),
        backgroundColor: const Color(0xFF1A237E),
      ),
      body: Padding(
        padding: const EdgeInsets.all(10.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Card(
              elevation: 4.0,
              shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12.0)),
              child: Container(
                padding: const EdgeInsets.symmetric(
                    vertical: 32.0, horizontal: 10.0),
                child: _isLoading
                    ? const Center(child: CircularProgressIndicator())
                    : (_errorMessage != null)
                        ? Text(_errorMessage!,
                            textAlign: TextAlign.center,
                            style: const TextStyle(color: Colors.red))
                        : (_kecamatan == null && _kota == null)
                            ? const Text(
                                'Tekan tombol untuk menampilkan lokasi.',
                                textAlign: TextAlign.center,
                              )
                            : Column(
                                crossAxisAlignment: CrossAxisAlignment.start,
                                children: [
                                  Text(
                                    'Kelurahan/Kecamatan: $_kecamatan',
                                    style: const TextStyle(
                                      fontSize: 16,
                                      fontWeight: FontWeight.bold,
                                    ),
                                  ),
                                  const SizedBox(height: 10),
                                  Text(
                                    'Kota: $_kota',
                                    style: const TextStyle(
                                      fontSize: 16,
                                      fontWeight: FontWeight.bold,
                                    ),
                                  ),
                                ],
                              ),
              ),
            ),
            const SizedBox(height: 40),
            ElevatedButton(
              onPressed: _isLoading ? null : _getLocation,
              style: ElevatedButton.styleFrom(
                backgroundColor: const Color(0xFF1A237E),
                padding: const EdgeInsets.symmetric(vertical: 16.0),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(12.0),
                ),
                elevation: 5.0,
              ),
              child: const Text(
                'TAMPILKAN LOKASI SAAT INI',
                style: TextStyle(
                  fontSize: 16,
                  color: Colors.white,
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
```

Pada method *build()* kita membuat tampilan antarmuka menggunakan widget *Scaffold* sebagai kerangka dasar halaman. Di bagian atas halaman terdapat *AppBar* yang menampilkan judul aplikasi *"Lokasi Saya"* dengan warna latar biru gelap. Seluruh konten halaman dibungkus dengan *Padding* untuk memberi jarak antara tepi layar dan elemen UI agar tampilan lebih rapi.

Di dalam *Padding* digunakan *Column* untuk menempatkan *widget* secara vertikal. *mainAxisAlignment* diatur ke *center* agar semua elemen berada di tengah layar secara vertikal, sedangkan *crossAxisAlignment* diatur ke *stretch* supaya tombol dan kartu informasi melebar menyesuaikan lebar layar.

Untuk menampilkan informasi lokasi, kita menggunakan widget *Card* dengan efek *elevation* untuk memberi bayangan dan RoundedRectangleBorder agar sudutnya membulat. Di dalam *Card* digunakan *Container* untuk menambahkan padding internal sehingga teks dan indikator tidak menempel ke tepi kartu.

Bagian konten kartu menyesuaikan kondisi aplikasi menggunakan logika *_isLoading*, *_errorMessage*, dan data lokasi. Jika *_isLoading* bernilai *true* maka akan muncul *CircularProgressIndicator* di tengah kartu untuk menunjukkan bahwa aplikasi sedang memproses pengambilan lokasi. Jika terjadi kesalahan, misalnya izin lokasi ditolak, *_errorMessage* akan menampilkan pesan error berwarna merah. Apabila lokasi belum diambil dan tidak ada error, akan muncul teks placeholder yang memberi instruksi kepada pengguna untuk menekan tombol. Setelah lokasi berhasil didapatkan, kartu menampilkan nama kecamatan dan kota yang diperoleh dari fungsi reverse geocoding.

Di bawah *Card* terdapat tombol *ElevatedButton* yang berfungsi untuk memicu pengambilan lokasi dengan memanggil metode *_getLocation()*. Tombol ini secara otomatis dinonaktifkan saat aplikasi sedang memproses lokasi (*_isLoading == true*) agar pengguna tidak menekan tombol berulang kali. Dengan pengaturan ini, UI menjadi interaktif dan responsif, serta memberikan umpan balik visual yang jelas kepada pengguna sesuai kondisi aplikasi saat ini.

j. Jalankan programnya. Kemudian amati dan catat output programnya. Adapun tampilannya sebagai berikut:





12.3. Kesimpulan

a. Aplikasi Flutter dapat menampilkan lokasi perangkat secara real time dengan memanfaatkan paket *geolocator* dan *geocoding*. Dengan kombinasi ini, koordinat GPS bisa langsung diubah menjadi informasi alamat seperti kecamatan dan kota.
b. Untuk mengakses lokasi aplikasi harus meminta izin dari pengguna. Penambahan *ACCESS_FINE_LOCATION* dan *ACCESS_COARSE_LOCATION* pada *AndroidManifest* memastikan aplikasi bisa mendapatkan akses GPS dengan benar.
c. *Widget Scaffold*, Card, dan *ElevatedButton* membantu membuat antarmuka yang rapi dan interaktif. Tampilan dapat menyesuaikan kondisi aplikasi, misalnya saat memuat data, terjadi kesalahan, atau data lokasi sudah tersedia.
d. Fungsi *_getLocation()* menggabungkan pengecekan status layanan lokasi, permintaan izin, pengambilan posisi, dan *reverse geocoding*. Dengan cara ini, aplikasi menjadi lebih stabil dan aman, karena semua kemungkinan error ditangani dengan baik.
e. Dengan *setState()* aplikasi secara otomatis memperbarui tampilan ketika data berubah. Setiap perubahan pada *_isLoading*, *_kecamatan*, *_kota*, atau *_errorMessage* langsung membuat *widget* terkait dibangun ulang, sehingga informasi yang ditampilkan selalu terbaru dan sesuai kondisi saat ini.

12.4. Tugas

Buatlah laporan hasil praktikum dari Modul 12 Akses Lokasi dengan GPS di Flutter yang berisi penjelasan mengenai proses pengambilan dan penampilan lokasi perangkat dalam aplikasi Flutter. Laporan harus mencakup uraian tentang tujuan praktikum, langkah-langkah praktikum yang telah dilakukan, serta penjelasan bagaimana fungsi *_getLocation()*, variabel *_isLoading*, *_kecamatan*, *_kota*, dan *_errorMessage* digunakan untuk menampilkan informasi lokasi secara dinamis dan responsif. Jelaskan juga bagaimana *setState()* berperan dalam memperbarui tampilan secara otomatis sesuai perubahan data.

Sertakan screenshot dari aplikasi yang menampilkan beberapa kondisi, antara lain: tampilan awal sebelum lokasi ditampilkan, saat aplikasi memproses pengambilan lokasi, lokasi berhasil ditampilkan, dan jika terjadi error. Setiap screenshot diberi keterangan singkat agar mudah dipahami.

Laporan disusun secara sistematis dan rapi sesuai format yang telah ditentukan, sehingga alur praktikum, penerapan kode program, serta hasil implementasi akses lokasi menggunakan GPS di Flutter dapat dijelaskan dengan jelas dan mudah dipahami.

12.5. Format Laporan

Laporan praktikum Modul 12 Akses Lokasi dengan GPS di Flutter disusun sesuai format berikut:

a. Cover

Berisi judul laporan, nama modul, serta informasi proyek atau aplikasi yang dibuat, termasuk tanggal praktikum.

b. Tujuan

Menjelaskan tujuan praktikum, yaitu memahami cara pengambilan lokasi perangkat menggunakan GPS di Flutter, penerapan paket *geolocator* dan *geocoding*, penanganan izin lokasi dan *error*, serta pembuatan antarmuka interaktif yang menampilkan informasi lokasi secara *real time*.

c. Langkah Kerja Singkat

Mencantumkan langkah-langkah praktikum secara ringkas, meliputi:

- Penambahan izin lokasi di *AndroidManifest.xml*.
- Penambahan dependensi pada file *pubspec.yam*l.
- Pembuatan UI menggunakan *Scaffold*, *Card*, dan *ElevatedButton*.
- Implementasi fungsi *_getLocation()* beserta logika pengecekan layanan lokasi, permintaan izin, pengambilan posisi, dan *reverse geocoding*.
- Penggunaan *setState()* untuk memperbarui tampilan berdasarkan perubahan data lokasi.

d. Screenshot

Menyertakan tangkapan layar aplikasi untuk berbagai kondisi, misalnya: tampilan awal sebelum lokasi ditampilkan, saat pemrosesan lokasi berlangsung, setelah lokasi berhasil ditampilkan, dan saat muncul pesan *error*. Setiap gambar diberi keterangan singkat untuk memudahkan pemahaman.

e. Kesimpulan

Memuat ringkasan temuan praktikum, termasuk bagaimana aplikasi menampilkan lokasi secara *real time*, penggunaan *setState()* untuk pembaruan UI otomatis, serta penanganan izin dan *error* selama pengambilan lokasi.